# 《算法设计与分析》
# 实验报告

班级 数据 231
姓名 艾洋
学号 235127

# 实验一、递归与分治

[1] 实验内容:

(1) 利用分治算法，编程实现最近点对问题，并进行时间复杂性分析。

(2) 可视化结果，验证结果的的着正确性。

[2] 实验目的:

(1) 深刻理解并掌握“分治算法”的设计思想;

(2) 提高应用“分治算法”设计技能;

(3) 理解这样一个观点: 用递归方法编写的问题解决程序具有结构清晰，可读性强等优点，且递归算法的设计比非递归算法的设计往往要容易一些，所以当问题本身是递归定义的，或者问题所涉及到的数据结构是递归定义的，或者是问题的解决方法是递归形式的时候，往往采用递归算法来解决。

[3] 程序清单:

```
import random
import time
import matplotlib.pyplot as plt


def generate_points(n):
    return [(random.uniform(1, 10), random.uniform(1, 10)) for _ in
range(n)]


def distance(p1, p2):
    return ((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2)**0.5
```

```
class BruteForceSolver:
    def __init__(self, points):
        self.points = points

    def solve(self):
        min_dist = float('inf')
        pair = None
        n = len(self.points)
        for i in range(n):
            for j in range(i + 1, n):
                dist = distance(self.points[i], self.points[j])
                if dist < min_dist:
                    min_dist = dist
                    pair = (self.points[i], self.points[j])
        return (min_dist, pair)

class DivideConquerSolver:
    def __init__(self, points):
        self.points = points

    def closest_pair_recursive(self, points_sorted):
        n = len(points_sorted)
        if n <= 3:
            return BruteForceSolver(points_sorted).solve()

        mid = n // 2
        left = points_sorted[:mid]
        right = points_sorted[mid:]
```

```
        d_left, pair_left = self.closest_pair_recursive(left)
        d_right, pair_right = self.closest_pair_recursive(right)

        d, pair = (d_left, pair_left) if d_left < d_right else
(d_right, pair_right)

        mid_x = points_sorted[mid][0]
        strip = [p for p in points_sorted if abs(p[0] - mid_x) < d]
        strip_sorted = sorted(strip, key=lambda x: x[1]) # 按 y 坐
标排序

        min_strip = d
        strip_pair = pair
        for i in range(len(strip_sorted)):
            for j in range(i + 1, min(i + 7, len(strip_sorted))):
                    current_dist = distance(strip_sorted[i],
strip_sorted[j])
                if current_dist < min_strip:
                    min_strip = current_dist
                                strip_pair = (strip_sorted[i],
strip_sorted[j])

        if min_strip < d:
        return (min_strip, strip_pair)
        else:
            return (d, pair)

    def solve(self):
        points_sorted = sorted(self.points, key=lambda x: x[0])
```

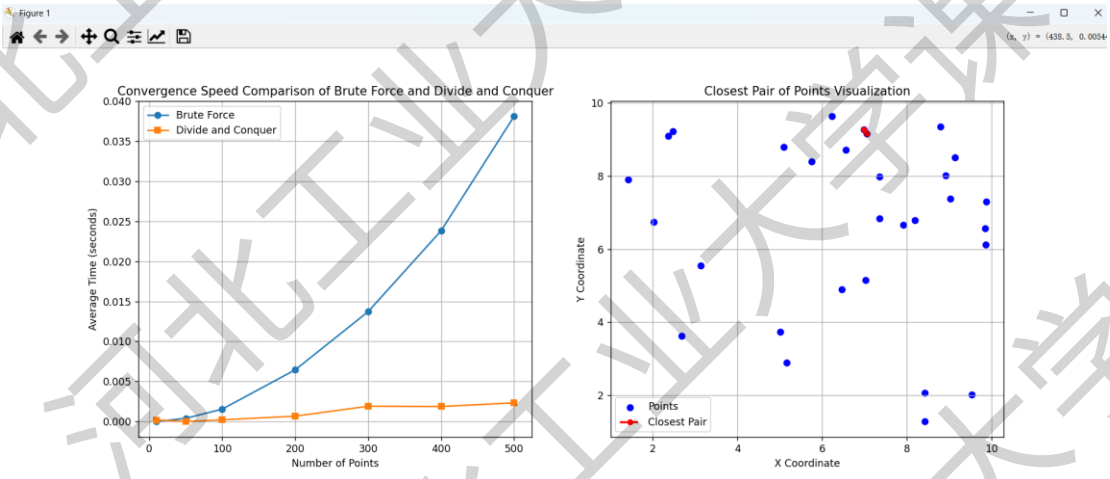
```
        return self.closest_pair_recursive(points_sorted)

def main():
    # 不同数据规模
    sizes = [10, 50, 100, 200, 300, 400, 500]
    times_bf = []
    times_dc = []

    # 多次运行求平均时间
    num_runs = 5
    for size in sizes:
        total_time_bf = 0
        total_time_dc = 0
        for _ in range(num_runs):
            points = generate_points(size)

            # 蛮力法测试
            start_bf = time.time()
            bf_solver = BruteForceSolver(points)
            min_dist_bf, pair_bf = bf_solver.solve()
            total_time_bf += time.time() - start_bf

            # 分治法测试
            start_dc = time.time()
            dc_solver = DivideConquerSolver(points)
            min_dist_dc, pair_dc = dc_solver.solve()
            total_time_dc += time.time() - start_dc
```

```
        avg_time_bf = total_time_bf / num_runs
        avg_time_dc = total_time_dc / num_runs

        times_bf.append(avg_time_bf)
        times_dc.append(avg_time_dc)

    # 生成 30 个随机点
    points = generate_points(30)

    # 蛮力法测试
    start_bf = time.time()
    bf_solver = BruteForceSolver(points)
    min_dist_bf, pair_bf = bf_solver.solve()
    time_bf = time.time() - start_bf

    # 分治法测试
    start_dc = time.time()
    dc_solver = DivideConquerSolver(points)
    min_dist_dc, pair_dc = dc_solver.solve()
    time_dc = time.time() - start_dc

    # 输出结果
    print("Brute Force Method:")
    print(f"Closest pair: {pair_bf[0]}, {pair_bf[1]}")
    print(f"Distance: {min_dist_bf:.6f}")
    print(f"Time taken: {time_bf:.10f} seconds\n")

    print("Divide and Conquer Method:")
```

```
    print(f"Closest pair: {pair_dc[0]}, {pair_dc[1]}")
    print(f"Distance: {min_dist_dc:.6f}")
    print(f"Time taken: {time_dc:.10f} seconds\n")

    # 验证结果一致性
    assert abs(min_dist_bf - min_dist_dc) < 1e-6, "Results from two methods do not match!"

    # 创建一个包含两个子图的画布
    fig, axes = plt.subplots(1, 2, figsize=(20, 6))

    # 绘制收敛速度图
    axes[0].plot(sizes, times_bf, label='Brute Force', marker='o')
    axes[0].plot(sizes, times_dc, label='Divide and Conquer', marker='s')
    axes[0].set_xlabel('Number of Points')
    axes[0].set_ylabel('Average Time (seconds)')
    axes[0].set_title('Convergence Speed Comparison of Brute Force and Divide and Conquer')
    axes[0].legend()
    axes[0].grid(True)

    # 绘制最近点对结果图
    x_all = [p[0] for p in points]
    y_all = [p[1] for p in points]
    axes[1].scatter(x_all, y_all, color='blue', label='Points')

    # 绘制最近点对
```

```
    p1, p2 = pair_bf
    axes[1].plot([p1[0], p2[0]], [p1[1], p2[1]],
                 color='red', linestyle='--', linewidth=2,
                 marker='o', markersize=5, label='Closest Pair')

    axes[1].set_xlabel('X Coordinate')
    axes[1].set_ylabel('Y Coordinate')
    axes[1].set_title('Closest Pair of Points Visualization')
    axes[1].legend()
    axes[1].grid(True)

    plt.show()

if __name__ == '__main__':
    main()
```

[4] 运行结果:



[5] 分析与思考:

在这次算法实验中，通过实现蛮力法和分治法来解决寻找最近点对的问题，并比较了这两种算法的性能。实验结果清晰地展示了分治法在处理大规模数据集时的效率优势，验证了其 O(n log n)时间复杂度相较于蛮力法 O(n^2)的优越性。此外，通过可视化算法性能和最近点对的结果，实验不仅直观地展示了算法的运行效果，也加深了对算法性能差异的理解。

这次实验引发了对算法优化和应用场景的深入思考。可以考虑探索并行计算在分治法中的应用，以进一步提高算法的运行效率。同时，实验中对结果的验证过程也强调了在算法设计和实现过程中，正确性和稳定性的重要性。这些思考将有助于在未来的学习和研究中，更加全面地评估和选择算法，以解决实际问题。

# 实验二 动态规划

[1] 实验内容:

(1) 利用动态规划算法编程求解 TSP 问题，并进行时间复杂性分析;

(2) 利用动态规划算法编程求解 0-1 背包问题，并进行时间复杂性分析。

[2] 实验目的:

(1) 熟练掌握动态规划思想及教材中相关经典算法。

(2) 使用动态规划法编程，求解 0/1 背包问题和 TSP 问题。

[3] 程序清单:

```
01 背包
import matplotlib.pyplot as plt
from tabulate import tabulate
# 设置 matplotlib 支持中文的字体
plt.rcParams['font.sans-serif'] = ['SimHei']  # 支持中文


def knapsack(n, weights, values, capacity):
    # 初始化动态规划表格
    dp = [[0]*(capacity+1) for _ in range(n+1)]
    selected = [0]*n

    # 创建可视化数据容器
    visual_data = [[" "*(n+2) for _ in range(capacity+1)] for _ in
range(n+1)]
```

```
    # 构建动态规划表
    for i in range(n+1):
        for w in range(capacity+1):
            if i == 0 or w == 0:
                dp[i][w] = 0
                visual_data[i][w] = '0'
            elif weights[i-1] <= w:
                dp[i][w] = max(dp[i-1][w],
                               dp[i-1][w-weights[i-1]] + values[i-
1])
                visual_data[i][w] = f"{dp[i][w]}←" + (
                    "上" if dp[i][w] == dp[i-1][w]
                    else f"取{i}"
                )
            else:
                dp[i][w] = dp[i-1][w]
                visual_data[i][w] = f"{dp[i][w]}←上"

    # 回溯选择过程
    w = capacity
    for i in range(n, 0, -1):
        if dp[i][w] != dp[i-1][w]:
            selected[i-1] = 1
            w -= weights[i-1]

    return dp[n][capacity], selected, visual_data

def visualize_table(data, items, capacity):
```

```
    # 创建表格头
    headers = ["物品\\容量"] + [str(i) for i in range(capacity+1)]

    # 创建表格行
    rows = []
    for i in range(len(data)):
        row = [f"物品{i}" if i>0 else "无物品"] + data[i]
        rows.append(row)

    # 使用 matplotlib 绘制表格
    plt.figure(figsize=(15, 5))
    ax = plt.gca()
    ax.axis('off')

    table = ax.table(cellText=rows,
                     colLabels=headers,
                     loc='center',
                     cellLoc='center')

    table.auto_set_font_size(False)
    table.set_fontsize(10)
    table.scale(1, 1.5)
    plt.title("动态规划过程可视化（箭头表示状态转移方向）")
    plt.show()

def visualize_selection(selected, weights, values):
    # 创建物品展示图
```

```
    fig, ax = plt.subplots(figsize=(10, 4))

    # 绘制背包
    ax.add_patch(plt.Rectangle((0.1, 0.2), 0.8, 0.6, fill=None,
lw=2, edgecolor='black'))
    ax.text(0.5, 0.9, "背包", ha='center', va='center',
fontsize=12)

    # 绘制物品
    positions = [(0.2 + i*0.15, 0.5) for i in range(len(selected))]
    for i, (s, w, v) in enumerate(zip(selected, weights, values)):
        color = 'green' if s else 'red'
        circle = plt.Circle(positions[i], 0.05, color=color,
zorder=2)
        ax.add_patch(circle)
        if s:
            # 给选中物品添加白色边框
            border = plt.Circle(positions[i], 0.05, color='white',
fill=False, lw=2, zorder=3)
            ax.add_patch(border)
        ax.text(positions[i][0]-0.02, positions[i][1]-0.1,
                f"{w}kg\n{v}$", fontsize=8, ha='center',
va='center')

    # 添加选中物品提示
    selected_items = [i+1 for i, s in enumerate(selected) if s]
    if selected_items:
        ax.text(0.5, 0.1, f"选中物品编号: {', '.join(map(str,
```

```
selected_items))}", ha='center', va='center', fontsize=10)

    ax.set_xlim(0, 1)
    ax.set_ylim(0, 1)
    ax.axis('off')
    plt.title("背包选择结果（绿色为选中物品）")
    plt.show()

if __name__ == "__main__":
    weights = [2, 2, 6, 5, 4]
    values = [6, 3, 5, 4, 6]
    capacity = 10

    max_value, selected, visual_data = knapsack(
        len(weights), weights, values, capacity
    )

    # 控制台输出
    print("动态规划表格: ")
    print(tabulate(visual_data,
                   headers=[str(i) for i in range(capacity+1)],
                   showindex="always",
                   tablefmt="fancy_grid"))

    print(f"\n最大价值: {max_value}")
    print("选中物品: ", [i+1 for i, s in enumerate(selected) if s])

    # 图形化展示
```

```
    visualize_table(visual_data, weights, capacity)
    visualize_selection(selected, weights, values)
```

TSP 问题

```
import math
from itertools import combinations
import matplotlib.pyplot as plt
import networkx as nx
from mpl_toolkits.mplot3d import Axes3D
import numpy as np

def tsp_dp(C):
    n = len(C)
    memo = {}  # 存储子问题的解
    all_cities = list(range(n))  # 包含所有城市的列表

    # 预处理: 从各个城市直接返回起点 0 的代价
    for city in range(n):
        if city == 0:  # 起点 0 不需要处理
            continue
        memo[(city, (0,))] = C[city][0]  # 直接返回起点 0 的代价

    # 动态规划填表, 处理包含 0 的子集
    def get_prev_subset(subset, next_city):
        return tuple([x for x in subset if x != next_city])
```

```
    #subset_size 为子集大小, subset 为当前子集
    for subset_size in range(1, n):  # 子集大小从 1 到 n-1, range 生成一个不包含 n 的整数序列
        for subset in combinations(all_cities, subset_size):# 生成所有子集
            if 0 not in subset:#跳过没有 0 的子集, 因为要从 0 出发
                continue
            subset = tuple(sorted(subset))  # 确保子集有序

            for current_city in all_cities:
                if current_city in subset:
                    continue  # 当前城市不能在子集中
                # 寻找所有可能的 next_city 进行状态转移
                min_cost = math.inf

                for next_city in subset:
                    if next_city == 0 and len(subset) > 1:#未遍历完
                        continue

                    # 前一个子集 (移除 next_city)
                        prev_subset = get_prev_subset(subset, next_city)
                    prev_key = (next_city, prev_subset)
                    if prev_key in memo:#如果前一个子问题的解已经
```

计算过

```
                        # 计算当前子问题的解
                            cost = C[current_city][next_city] +
memo[prev_key]
                        if cost < min_cost:
                            min_cost = cost
                if min_cost != math.inf:
                    memo[(current_city, subset)] = min_cost

    # 计算最终结果：从 0 出发，经过所有城市后回到 0
    full_subset = tuple(all_cities)
    min_total = math.inf
    for last_city in all_cities[1:]:# 遍历所有城市，排除 0
        prev_subset = tuple([x for x in full_subset if x !=
last_city])# 前一个子集 (移除 last_city)
        key = (last_city, prev_subset)
        if key in memo:
            cost = C[0][last_city] + memo[key]
            if cost < min_total:
                min_total = cost
    return min_total, memo

def draw_dp_table(memo, n, C, min_total):
    # 生成所有包含 0 的子集并按长度排序
    subsets = []
    for k in range(n + 1):
        for subset in combinations(range(n), k):
```

```
            if 0 in subset:
                subset = tuple(sorted(subset))
                subsets.append(subset)
    subsets = sorted(subsets, key=lambda x: (len(x), x))

    # 构建表格列名
    columns = ['City \\ Subset']
    for subset in subsets:
        subset_str = "{" + ",".join(map(str, subset)) + "}"
        columns.append(subset_str)

    # 填充表格数据
    table_data = []
    full_subset = tuple(range(n))
    for city in range(n):
        row = [f'City {city}']
        for subset in subsets:
            if subset == full_subset:
                # 仅在 City 0 对应的格子填充最小值
                if city == 0:
                    row.append(str(min_total))
                else:
                    row.append('')
            else:
                key = (city, subset)
                value = memo.get(key, '')
                row.append(str(value) if value != '' else '')
        table_data.append(row)
```

```
    # 绘制表格
    fig, ax = plt.subplots(figsize=(18, 4))
    ax.axis('tight')
    ax.axis('off')
        table = ax.table(cellText=table_data, colLabels=columns,
loc='center', cellLoc='center')
    table.auto_set_font_size(False)
    table.set_fontsize(8)
    plt.title("TSP DP Table")
    plt.show(block=False)

def draw_cost_graph(C):
    # 使用有向图
    G = nx.DiGraph()
    n = len(C)
    if n != 4:
        print("当前城市数量不是 4，无法绘制正四面体。")
        return
    # 添加节点
    G.add_nodes_from(range(n))
    # 添加边和权重
    for i in range(n):
        for j in range(n):
            if i != j and C[i][j] != math.inf:
                G.add_edge(i, j, weight=C[i][j])

    # 正四面体顶点坐标
```

```
    pos_3d = {
        0: (0, 0, 0),
        1: (1, 0, 0),
        2: (0.5, np.sqrt(3) / 2, 0),
        3: (0.5, np.sqrt(3) / 6, np.sqrt(6) / 3)
    }

    # 定义四种颜色
    colors = ['r', 'g', 'b', 'y']

    # 创建三维图形
    fig = plt.figure()
    ax = fig.add_subplot(111, projection='3d')

    # 绘制节点
    for node in G.nodes():
        x, y, z = pos_3d[node]
        # 调整节点大小
        ax.scatter(x, y, z, c='b', s=100)
        # 调整序号文字位置，使其稍微远离节点
        offset = 0.05
         ax.text(x + offset, y + offset, z + offset, str(node),
color='red', fontsize=10)

    # 绘制有向边
    for edge in G.edges():
        start = pos_3d[edge[0]]
```

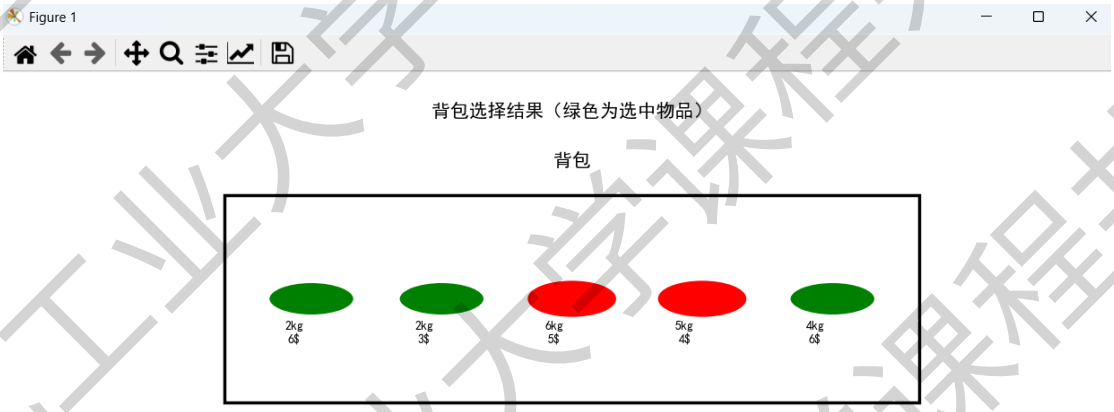
```
        end = pos_3d[edge[1]]
        start_node = edge[0]
        # 根据起点选择颜色
        edge_color = colors[start_node % len(colors)]
        ax.plot([start[0], end[0]], [start[1], end[1]], [start[2],
end[2]], c=edge_color)

        # 让权值靠近起点
        weight_pos_x = start[0] + 0.2 * (end[0] - start[0])
        weight_pos_y = start[1] + 0.2 * (end[1] - start[1])
        weight_pos_z = start[2] + 0.2 * (end[2] - start[2])
        weight = G[edge[0]][edge[1]]['weight']
              ax.text(weight_pos_x,  weight_pos_y,  weight_pos_z,
str(weight), color='green')

    ax.set_title("3D Cost Graph for TSP")
    plt.show(block=False)

# 示例调用
C = [
    [math.inf, 3, 6, 7],
    [5, math.inf, 2, 3],
    [6, 4, math.inf, 2],
    [3, 7, 5, math.inf]
]

min_cost, memo = tsp_dp(C)
print("最短路径代价:", min_cost)
```

```
draw_dp_table(memo, len(C), C, min_cost)  # 传入最短路径代价
draw_cost_graph(C)
plt.show()
```

[4] 运行结果:

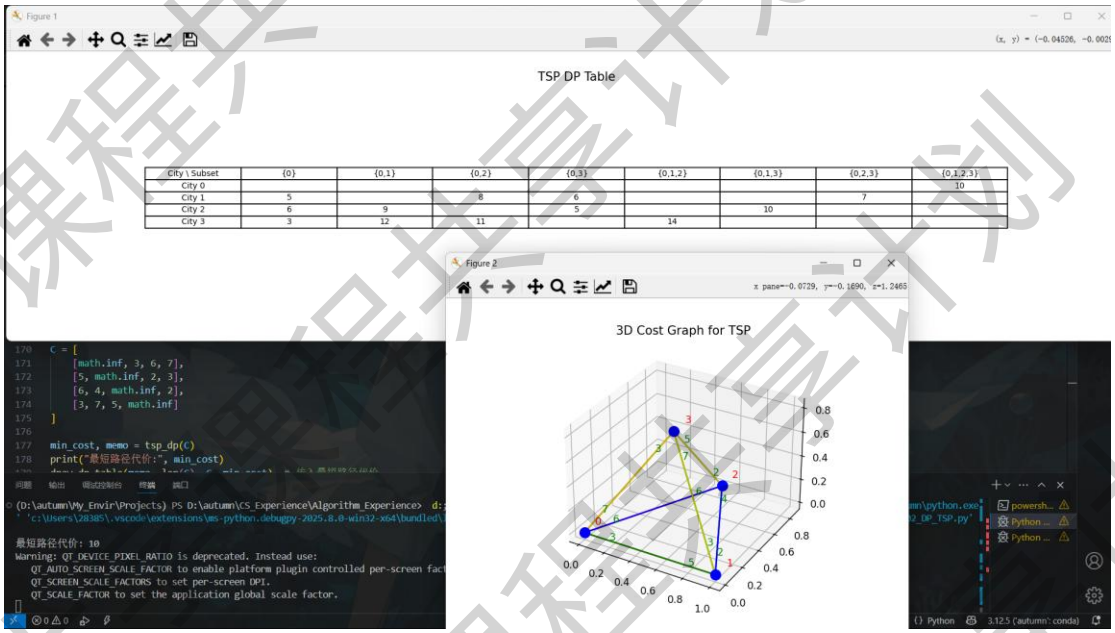
01 背包


| 物品\容量 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 无物品 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 物品1 | 0 | 0←上 | 6←取1 | 6←取1 | 6←取1 | 6←取1 | 6←取1 | 6←取1 | 6←取1 | 6←取1 | 6←取1 |
| 物品2 | 0 | 0←上 | 6←上 | 6←上 | 9←取2 | 9←取2 | 9←取2 | 9←取2 | 9←取2 | 9←取2 | 9←取2 |
| 物品3 | 0 | 0←上 | 6←上 | 6←上 | 9←上 | 9←上 | 9←上 | 9←上 | 11←取3 | 11←取3 | 14←取3 |
| 物品4 | 0 | 0←上 | 6←上 | 6←上 | 9←上 | 9←上 | 9←上 | 10←取4 | 11←上 | 13←取4 | 14←上 |
| 物品5 | 0 | 0←上 | 6←上 | 6←上 | 9←上 | 9←上 | 12←取5 | 12←取5 | 15←取5 | 15←取5 | 15←取5 |




选中物品编号: 1, 2, 5

TSP 问题




[5] 分析与思考:

在背包问题的实验中,通过实现动态规划算法,成功解决了 0-1 背包问题,并可视化了动态规划表和选择结果。实验结果表明,动态规划方法能够有效地找到背包问题的最大价值解,并通过可视化过程加深了对算法执行过程的理解。实验中,通过构建动态规划表和回溯选择过程,直观地展示了算法的工作原理和状态转移过程。

在旅行商问题(TSP)的实验中,通过实现动态规划算法,成功求解了给定城市间最短路径问题,并可视化了动态规划表和成本图。实验结果验证了动态规划方法在解决 TSP 问题上的有效性,并通过可视化过程直观地展示了算法的执行过程和状态转移。

# 实验三　贪心算法

[1] 实验内容:

(1) TSP 问题最近邻点策略，并对算法进行时间复杂性分析。

[2] 实验目的:

(1) 掌握贪心算法求解问题的一般特征和步骤;

(2) 使用贪心算法编程，求解 TSP 问题。

[3] 程序清单:

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.ticker import MaxNLocator


# 给定的距离矩阵
C = np.array([
    [np.inf, 3, 3, 2, 6],
    [3, np.inf, 7, 3, 2],
    [3, 7, np.inf, 2, 5],
    [2, 3, 2, np.inf, 3],
    [6, 2, 5, 3, np.inf]
])

# 城市坐标（假设出来的）
coords = np.array([
    [0, 0],
```

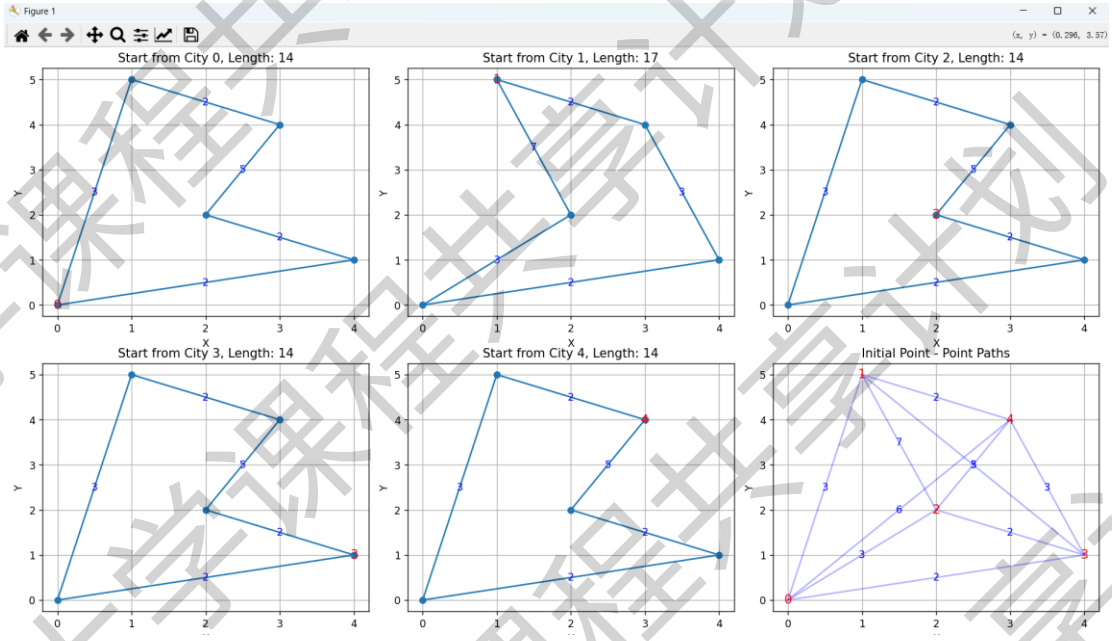
```
    [1, 5],
    [2, 2],
    [4, 1],
    [3, 4]
])

def nearest_neighbor_tsp(C, coords, start_city):
    n = len(C)
    visited = [False] * n
    path = [start_city]  # 从指定城市开始
    visited[start_city] = True
    current_city = start_city

    while len(path) < n:
        last_city = path[-1]
        next_city = None
        min_dist = np.inf

        for i in range(n):
            if not visited[i] and C[last_city][i] < min_dist:
                min_dist = C[last_city][i]
                next_city = i

        path.append(next_city)
        visited[next_city] = True
        current_city = next_city

    path.append(start_city)  # 返回起始城市
    return path
```

```
def calculate_path_length(C, path):
    length = 0
    for i in range(len(path) - 1):
        length += C[path[i]][path[i + 1]]
    return length

# 创建一个 2x3 的子图布局
fig, axes = plt.subplots(2, 3, figsize=(15, 10))
axes = axes.flatten()

# 计算从每个城市出发的路径并绘图
for start_city in range(len(C)):
    path = nearest_neighbor_tsp(C, coords, start_city)
    path_length = calculate_path_length(C, path)
    path = np.array(path)
    coords_path = coords[path]

    ax = axes[start_city]
    ax.plot(coords_path[:, 0], coords_path[:, 1], 'o-')
        ax.plot([coords_path[0, 0], coords_path[-1, 0]],
[coords_path[0, 1], coords_path[-1, 1]], 'k-')
    # 路径长度取整显示
        ax.set_title(f"Start from City {start_city}, Length:
{int(path_length)}")
    ax.set_xlabel("X")
    ax.set_ylabel("Y")
    ax.grid(True)
```

```
    # 标注起始点序号
    start_coord = coords[start_city]
        ax.text(start_coord[0],  start_coord[1],  str(start_city),
color='red', fontsize=12, ha='center', va='center')

    # 在每段路径上标注长度，取整显示
    for i in range(len(path) - 1):
        city1 = path[i]
        city2 = path[i + 1]
        dist = C[city1][city2]
        x_mid = (coords[city1][0] + coords[city2][0]) / 2
        y_mid = (coords[city1][1] + coords[city2][1]) / 2
            ax.text(x_mid,  y_mid,  f"{int(dist)}",  color='blue',
fontsize=10, ha='center', va='center')

    # 设置横轴刻度为整数
    ax.xaxis.set_major_locator(MaxNLocator(integer=True))

# 第六张图显示最初点与点之间的路径
ax = axes[-1]
for i in range(len(coords)):
    for j in range(i + 1, len(coords)):
            ax.plot([coords[i][0],  coords[j][0]],  [coords[i][1],
coords[j][1]], 'b-', alpha=0.3)
        dist = C[i][j]
        x_mid = (coords[i][0] + coords[j][0]) / 2
        y_mid = (coords[i][1] + coords[j][1]) / 2
```

```
        # 标注长度取整显示
            ax.text(x_mid, y_mid, f"{int(dist)}", color='blue',
fontsize=10, ha='center', va='center')

# 标注城市序号
for i, coord in enumerate(coords):
    ax.text(coord[0], coord[1], str(i), color='red', fontsize=12,
ha='center', va='center')

# 修改最后一个子图的标题，去掉总长度信息
ax.set_title("Initial Point - Point Paths")
ax.set_xlabel("X")
ax.set_ylabel("Y")
ax.grid(True)
# 设置最后一个子图横轴刻度为整数
ax.xaxis.set_major_locator(MaxNLocator(integer=True))

plt.tight_layout()
plt.show()
```

[4] 运行结果:




[5] 分析与思考:

在本次实验中，通过实现最近邻算法解决了旅行商问题（TSP），并利用 matplotlib 库对结果进行了可视化。实验中，从每个城市作为起点计算了旅行路径，并展示了路径长度和城市间的距离。通过这种可视化方法，可以直观地比较不同起点下路径的差异，以及理解最近邻算法在解决 TSP 问题时的行为模式。实验结果表明，尽管最近邻算法实现简单，计算速度快，但它并不总是能得到最优解，而是容易陷入局部最优，这一点在可视化结果中得到了体现。

通过这次实验，我对旅行商问题有了更深入的理解，特别是对最近邻算法的局限性有了新的认识。此外，实验也让我意识到，对于 TSP 这类 NP 难问题，算法的选择和优化需要根据具体问题的特性和求解需求来决定，这在解决实际问题时尤为重要。

# 实验四　回溯法

[1] 实验内容:

（1） 利用回溯法编程求解 0-1 背包问题，并对算法进行时间复杂性分析;

（2） 利用回溯法编程求解 TSP 问题，并对算法进行时间复杂性分析。

[2] 实验目的:

（1）掌握回溯法的设计思想;

（2）掌握解空间树的构造方法，以及在求解过程中如何存储求解路径;

（3）考察回溯法求解问题的有效程度。

[3] 程序清单:

01 背包

```
import matplotlib.pyplot as plt
import matplotlib.animation as animation
import networkx as nx
import numpy as np


class Item:
    def __init__(self, weight, value):
        self.weight = weight
        self.value = value


def bag_backtracking(items, capacity):
    n = len(items)
    max_value = 0
    best_combination = []  # 记录最优解的物品组合
    G = nx.DiGraph()  # 创建有向图存储搜索树
```

```
    node_id = 0
    G.add_node(node_id, label=f"Start\nw: 0\nv: 0")
    frames = []

    def backtrack(index, current_weight, current_value, current_combination, parent_id):
        nonlocal max_value, best_combination, node_id

        if index == n:  # 遍历完所有物品
            if current_value > max_value:
                max_value = current_value
                best_combination = current_combination.copy()
            return

        # 记录当前状态，包含物品组合、当前重量、当前价值
        frame = (current_combination.copy(), current_weight, current_value)
        frames.append(frame)

        # 选当前物品（左枝干）
        if current_weight + items[index].weight <= capacity:
            new_node_id = node_id + 1
            # 用 w 和 v 表示重量和价值
            G.add_node(new_node_id, label=f"w: {current_weight + items[index].weight}\nv: {current_value + items[index].value}")
            G.add_edge(parent_id, new_node_id, label='1')  # 标记边为 1 表示选择
```

```
            node_id = new_node_id
            current_combination.append(index)  # 选择当前物品
                    backtrack(index + 1, current_weight +
items[index].weight,     current_value     +     items[index].value,
current_combination, new_node_id)
            current_combination.pop()

        # 不选当前物品（右枝干）
        new_node_id = node_id + 1
        # 用 w 和 v 表示重量和价值
          G.add_node(new_node_id, label=f"w: {current_weight}\nv:
{current_value}")
         G.add_edge(parent_id, new_node_id, label='0')  # 标记边为
0 表示不选择
        node_id = new_node_id
          backtrack(index + 1, current_weight, current_value,
current_combination, new_node_id)  # 不选择当前物品

    backtrack(0, 0, 0, [], 0)

    # 提前计算所有节点的位置
    pos = nx.nx_agraph.graphviz_layout(G, prog='dot')

    node_labels = nx.get_node_attributes(G, 'label')
    edge_labels = nx.get_edge_attributes(G, 'label')

    fig, ax = plt.subplots(figsize=(12, 8))
```

```
    # 绘制完整的搜索树，先隐藏节点和边
    nodes = nx.draw_networkx_nodes(G, pos, node_color='lightblue',
ax=ax, alpha=0)
    edges = nx.draw_networkx_edges(G, pos, ax=ax, alpha=0)
    nx.draw_networkx_labels(G, pos, labels=node_labels, ax=ax)
     nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels,
ax=ax)

    def update(frame):
        current_combination, current_weight, current_value = frame
        ax.clear()
        # 重新绘制完整的搜索树
                    nodes = nx.draw_networkx_nodes(G, pos,
node_color='lightblue', ax=ax)
        edges = nx.draw_networkx_edges(G, pos, ax=ax)
        nx.draw_networkx_labels(G, pos, labels=node_labels, ax=ax)
                            nx.draw_networkx_edge_labels(G, pos,
edge_labels=edge_labels, ax=ax)

        ax.set_title("0 - 1 Bag Backtracking Search Tree")
        # 用 w 和 v 表示重量和价值
          ax.text(0.02, 0.95, f"w: {current_weight}/{capacity}",
transform=ax.transAxes)
                ax.text(0.02, 0.90, f"v: {current_value}",
transform=ax.transAxes)

        plt.tight_layout()
```

```
        ani = animation.FuncAnimation(fig, update, frames=frames,
interval=500, repeat=False)
    plt.show()

    return max_value, best_combination

# 定义物品和背包容量
weights = [2, 2, 6, 5, 4]
values = [6, 3, 5, 4, 6]
capacity = 10

# 创建物品对象
items = [Item(w, v) for w, v in zip(weights, values)]

# 调用回溯法求解
max_value, best_combination = bag_backtracking(items, capacity)

# 输出结果
print("最大价值:", max_value)
print("最优组合 (物品索引) :", best_combination)
```

```
TSP 问题
import matplotlib.pyplot as plt
import matplotlib.animation as animation
import networkx as nx
```

```
def tsp_backtracking(C, n, current_city, path, min_cost, best_path, G, node_id, parent_id, frames):
    if len(path) == n:
        # 计算回到起点的距离
        if path:  # 确保 path 不为空
            return_cost = C[path[-1]][0]
            total_cost = return_cost + sum(C[path[i - 1]][path[i]] for i in range(1, n))
            if total_cost < min_cost[0]:
                min_cost[0] = total_cost
                best_path[0] = path + [0]
            # 记录当前状态
            frames.append((G.copy(), path.copy(), total_cost))
        return node_id

    # 计算当前路径的代价-
    current_cost = sum(C[path[i - 1]][path[i]] for i in range(1, len(path)))
    # 剪枝: 如果当前代价已经超过已知最小代价, 停止搜索该分支
    if current_cost >= min_cost[0]:
        return node_id

    new_node_id = node_id
    for next_city in range(n):  # 修正遍历范围
        if next_city not in path:
            new_node_id += 1
            # 计算添加下一个城市后的当前代价
```

```
            next_cost = current_cost + C[path[-1]][next_city] if
path else 0
                        G.add_node(new_node_id,    label=f"City:
{next_city}\nCost: {next_cost}")
                            G.add_edge(parent_id,    new_node_id,
label=f"{next_city}")
            path.append(next_city)
             new_node_id = tsp_backtracking(C, n, next_city, path,
min_cost, best_path, G, new_node_id, new_node_id, frames)
            path.pop()
            # 记录当前状态
            if path:  # 确保 path 不为空
                frames.append((G.copy(), path.copy(), sum(C[path[i
- 1]][path[i]] for i in range(1, len(path)))))
    return new_node_id


def solve_tsp(C):
    n = len(C)
    min_cost = [float('inf')]  # 存储最小代价
    best_path = [[]]  # 存储最佳路径
    G = nx.DiGraph()
    G.add_node(0, label=f"City: 0\nCost: 0")
    node_id = 0
    frames = []
    tsp_backtracking(C, n, 0, [0], min_cost, best_path, G, node_id,
0, frames)
```

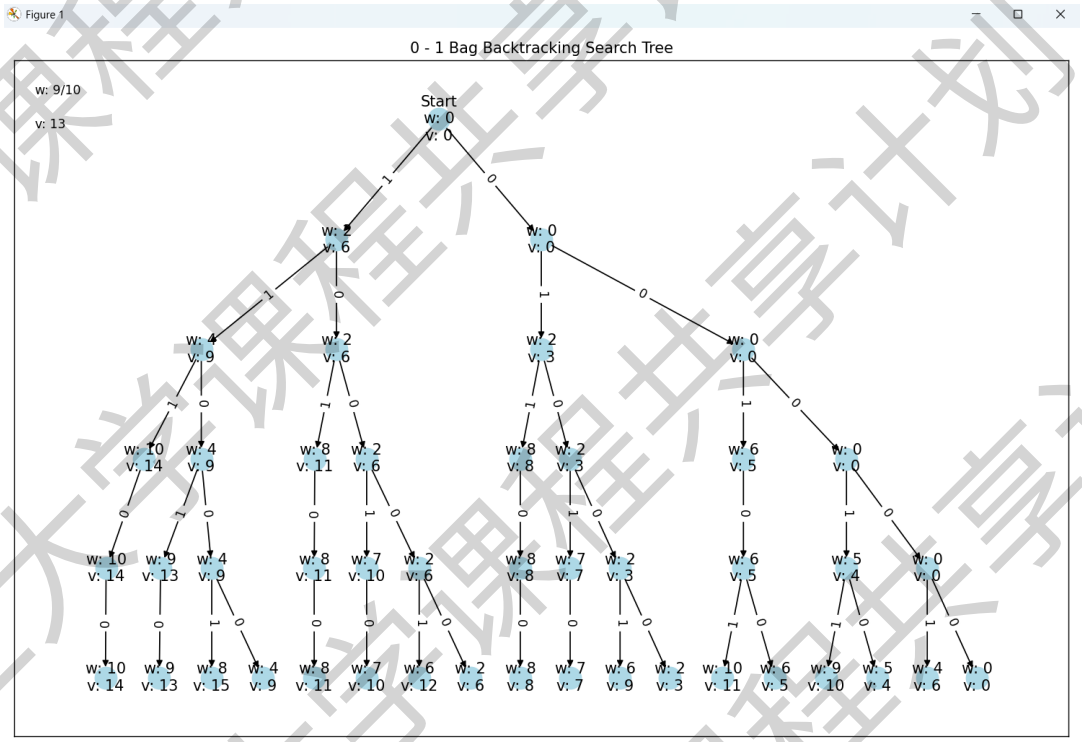
```
    # 提前计算所有节点的位置
    pos = nx.nx_agraph.graphviz_layout(G, prog='dot')

    node_labels = nx.get_node_attributes(G, 'label')
    edge_labels = nx.get_edge_attributes(G, 'label')

    fig, ax = plt.subplots(figsize=(12, 8))

    def update(frame):
        G, path, cost = frame
        ax.clear()
        # 重新绘制完整的搜索树
                    nodes = nx.draw_networkx_nodes(G, pos,
node_color='lightblue', ax=ax)
        edges = nx.draw_networkx_edges(G, pos, ax=ax)
        nx.draw_networkx_labels(G, pos, labels=node_labels, ax=ax)
                        nx.draw_networkx_edge_labels(G, pos,
edge_labels=edge_labels, ax=ax)

        ax.set_title("TSP Backtracking Search Tree")
                ax.text(0.02, 0.95, f"Current Path: {path}",
transform=ax.transAxes)
                ax.text(0.02, 0.90, f"Current Cost: {cost}",
transform=ax.transAxes)

        plt.tight_layout()

      ani = animation.FuncAnimation(fig, update, frames=frames,
interval=500, repeat=False)
```

```
        plt.show()

        return min_cost[0], best_path[0]

# 代价矩阵
C = [
        [float('inf'), 3, 6, 7],
        [5, float('inf'), 2, 3],
        [6, 4, float('inf'), 2],
        [3, 7, 5, float('inf')]
]

# 求解 TSP 问题
min_cost, best_path = solve_tsp(C)

# 输出结果
print("最小代价:", min_cost)
print("最佳路径:", best_path)
```

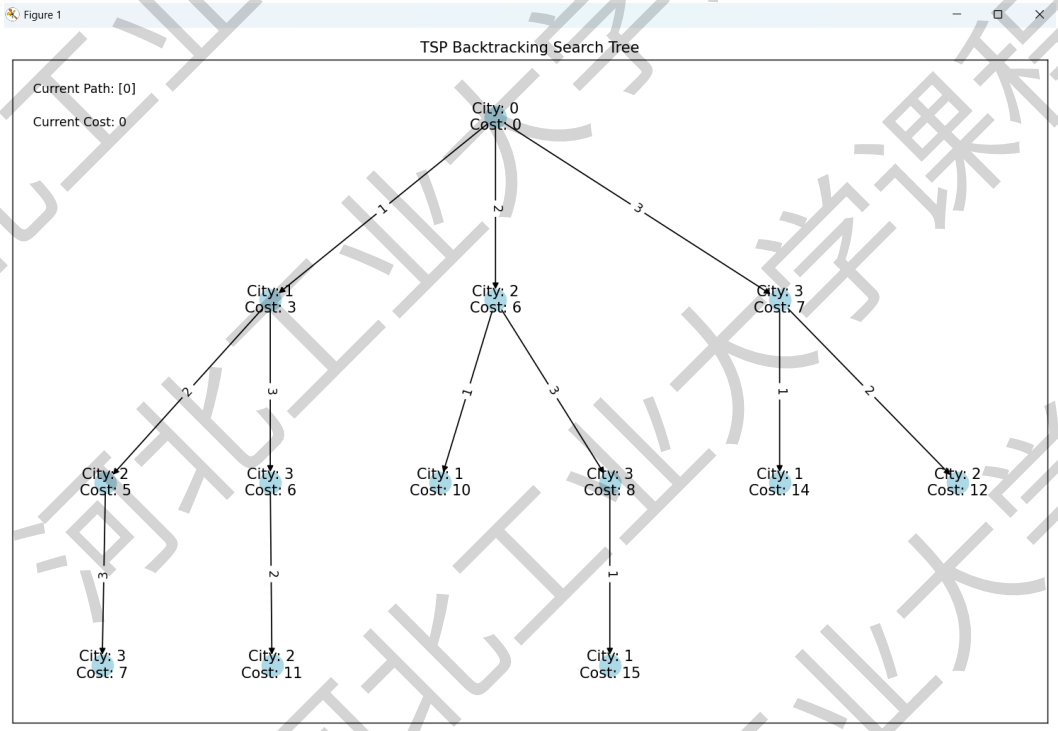
[4] 运行结果：

[5]

01 背包




TSP 问题



[5] 分析与思考:

在背包问题的实验中，通过使用回溯法探索所有可能的物品组合，并利用动态规划和图论库 networkx 来构建搜索树和可视化搜索过程。实验成功展示了算法如何逐步构建解决方案，并最终找到在给定容量限制下的最大价值组合。实验结果不仅验证了算法的有效性，而且通过动画展示了算法的探索过程，这有助于理解算法的工作原理和状态转移过程。此外，实验还引发了对算法优化的思考，例如，如何减少搜索空间和提高算法效率，这对于解决实际应用中的类似问题具有重要意义。

在旅行商问题（TSP）的实验中，通过实现回溯法来寻找最短路径，并利用 networkx 库和 matplotlib 库进行路径搜索过程的可视化。实验结果展示了算法如何通过逐步构建和评估路径来找到最优解，并通过动画形式直观地呈现了搜索过程。这不仅有助于理解算法的执行过程，也有助于发现算法的优化空间，例如通过剪枝策略减少不必要的搜索。实验还引发了对算法应用场景的思考，如在物流、旅游等领域中如何有效地规划路径以降低成本。这些思考对于算法的实际应用和进一步研究具有重要价值。